SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: MLA03
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO5: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr DUMPALA PRASANTH

Code of Conduct:

I (K Varshan / 192325089*) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

K Varshan
Signature of the student:

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Simulation-Based Assessment Question

Duration: 1hr

1. Simulate a **Hierarchical Reinforcement Learning (HRL)** agent for autonomous warehouse navigation.

   (5 Marks)

2. Implement a **Hierarchical Abstract Machine (HAM)** to control a robot performing sequential tasks.

   (5 Marks)

3. Develop the **MAXQ Value Decomposition** algorithm for a taxi navigation environment.

   (5 Marks)

4. Simulate a **Meta-Reinforcement Learning** agent that rapidly adapts to multiple GridWorld environments.

   (5 Marks)

5. Design a **Multi-Agent Reinforcement Learning (MARL)** system for cooperative robot path planning.

   (5 Marks)

6. Implement a **competitive Multi-Agent RL** environment for autonomous drone navigation.

   (5 Marks)

7. Simulate a **Partially Observable Markov Decision Process (POMDP)** for autonomous vehicle navigation under limited visibility.

(5 Marks)

8. Design an **ethical Reinforcement Learning** simulation for autonomous decision-making while satisfying safety constraints.

(5 Marks)

9. Develop an RL-based **adaptive traffic signal control** system and evaluate its performance.

10. Simulate an RL-based **smart energy management system** for optimizing power distribution in a microgrid.. **.**

(5 Marks)

ANSWERS:

1. Hierarchical Reinforcement Learning (HRL) for Autonomous Warehouse Navigation

Concept

Hierarchical Reinforcement Learning divides a complex task into smaller subtasks. In warehouse navigation, a high-level controller can select goals such as "go to shelf," "pick item," and "go to packing area," while a low-level controller selects movement actions such as up, down, left, and right.

Simulation

```
import random

grid = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 0, 0],
    [0, 0, 0, 0, 0],
    [0, 0, 1, 1, 0],
    [0, 0, 0, 0, 0]
]

position = [0, 0]
goals = [[0, 4], [4, 4]]

def move(pos, action):
    r, c = pos
    if action == "down": r += 1
    if action == "up": r -= 1
    if action == "right": c += 1
    if action == "left": c -= 1

    if 0 <= r < 5 and 0 <= c < 5 and grid[r][c] == 0:
```

```
        return [r, c]
    return pos

for goal in goals:
    print("High-level goal:", goal)

    while position != goal:
        actions = ["up", "down", "left", "right"]
        best = min(actions,
                   key=lambda a: abs(move(position,a)[0]-goal[0]) +
                                 abs(move(position,a)[1]-goal[1]))
        position = move(position, best)
        print("Robot position:", position)

print("Warehouse task completed")
```

Application

The HRL agent first chooses a sub-goal and then uses low-level actions to reach that goal.

Analysis

HRL reduces the complexity of warehouse navigation because the agent does not learn the entire task as one large problem. Subtasks can be reused, improving learning efficiency and scalability.

Expected Result

The robot moves through the warehouse toward the required locations while avoiding blocked cells.

2. Hierarchical Abstract Machine (HAM) for Sequential Robot Tasks

Concept

A Hierarchical Abstract Machine (HAM) represents a task using a hierarchy of states and actions. It is suitable when a robot must perform tasks in a fixed sequence.

Example:

Start → Move → Pick → Move → Drop → Finish

Simulation

```
class RobotHAM:
    def __init__(self):
        self.state = "START"

    def run(self):
        while self.state != "FINISH":
            if self.state == "START":
                print("Robot started")
                self.state = "MOVE_TO_OBJECT"

            elif self.state == "MOVE_TO_OBJECT":
                print("Robot moved to object")
                self.state = "PICK"

            elif self.state == "PICK":
                print("Object picked")
                self.state = "MOVE_TO_TARGET"

            elif self.state == "MOVE_TO_TARGET":
                print("Robot moved to target")
                self.state = "DROP"
```

```
        elif self.state == "DROP":
            print("Object dropped")
            self.state = "FINISH"

    print("Task completed")

robot = RobotHAM()
robot.run()
```

Application

The HAM restricts the robot to valid task sequences and provides a hierarchical structure for complex actions.

Analysis

Compared with learning every possible action sequence, HAM reduces the search space by specifying the task structure. This improves efficiency and prevents invalid sequences such as dropping an object before picking it.

Expected Result

The robot completes the sequential task:

Start → Move → Pick → Move → Drop → Finish

3. MAXQ Value Decomposition for Taxi Navigation

Concept

MAXQ is a hierarchical reinforcement learning algorithm that decomposes a complex task into smaller subtasks. In a taxi environment, the main task can be divided into:

Navigate to passenger

Pick up passenger

Navigate to destination

Drop passenger

Simulation

```
import random

states = ["GoPassenger", "Pickup", "GoDestination", "Drop"]
actions = ["move", "wait"]

Q = {state: {action: 0 for action in actions} for state in states}

alpha = 0.5
gamma = 0.9

for episode in range(100):
    for state in states:
        action = random.choice(actions)

        if state == "GoPassenger":
            reward = 1
        elif state == "Pickup":
            reward = 5
        elif state == "GoDestination":
            reward = 2
        else:
            reward = 10

        old = Q[state][action]
```

```
Q[state][action] = old + alpha * (reward + gamma * max(Q[state].values()) - old)
```

```
for state in states:
    print(state, Q[state])
```

Application

The value function is decomposed among subtasks instead of learning only one value for the entire taxi task.

Analysis

MAXQ improves interpretability and learning efficiency because each subtask has its own value information. If navigation changes, the navigation subtask can be improved without completely relearning passenger pickup and drop-off.

Expected Result

The learned values identify actions that provide higher rewards for completing taxi subtasks.

4. Meta-Reinforcement Learning for Multiple GridWorld Environments

Concept

Meta-Reinforcement Learning (Meta-RL) enables an agent to learn how to learn. Instead of training for only one environment, the agent experiences several GridWorld environments and learns to adapt quickly to a new one.

Simulation

```
import random

environments = [
    {"goal": (4, 4)},
```

```
    {"goal": (0, 4)},
    {"goal": (4, 0)}
]

for episode in range(10):
    env = random.choice(environments)
    position = [0, 0]
    goal = env["goal"]

    for step in range(20):
        if tuple(position) == goal:
            print("Goal reached in", step, "steps")
            break

        actions = [
            [1, 0], [-1, 0],
            [0, 1], [0, -1]
        ]

        action = min(
            actions,
            key=lambda a:
            abs(position[0] + a[0] - goal[0]) +
            abs(position[1] + a[1] - goal[1])
        )

        position[0] += action[0]
        position[1] += action[1]

        position[0] = max(0, min(4, position[0]))
```

```
    position[1] = max(0, min(4, position[1]))
```

Application

The agent encounters different goal locations and learns a general navigation strategy.

Analysis

Meta-RL is useful when environments change frequently. Rather than training from zero for every new environment, the agent can adapt using knowledge acquired from previous environments.

Expected Result

The agent reaches different goals with fewer adaptation steps.

5. Multi-Agent Reinforcement Learning for Cooperative Robot Path Planning

Concept

Multi-Agent Reinforcement Learning (MARL) involves multiple agents learning simultaneously. For cooperative robot path planning, robots should reach their destinations while avoiding collisions.

Simulation

```
import random

robots = [
    {"pos": [0, 0], "goal": [4, 4]},
    {"pos": [4, 0], "goal": [0, 4]}
]

for step in range(10):
```

```
    for robot in robots:
        r, c = robot["pos"]
        gr, gc = robot["goal"]

        if r < gr:
            r += 1
        elif r > gr:
            r -= 1
        elif c < gc:
            c += 1
        elif c > gc:
            c -= 1

        robot["pos"] = [r, c]

    if robots[0]["pos"] == robots[1]["pos"]:
        print("Collision detected - changing path")
        robots[1]["pos"][1] += 1

    print("Robot positions:", [r["pos"] for r in robots])

    if all(r["pos"] == r["goal"] for r in robots):
        print("All robots reached goals")
        break
```

Application

Each robot acts as an agent and considers its own goal while coordinating with other robots.

Analysis

The reward can include:

Positive reward: reaching the destination

Negative reward: collision

Negative reward: unnecessary movement

Positive reward: successful coordination

This encourages safe and efficient paths.

Expected Result

Multiple robots reach their targets while reducing collisions and unnecessary movements.

6. Competitive Multi-Agent RL for Autonomous Drone Navigation

Concept

In competitive MARL, multiple agents have conflicting objectives. For autonomous drones, each drone may attempt to reach a target while avoiding other drones.

Simulation

```
import random

drones = [
    {"pos": [0, 0], "goal": [4, 4]},
    {"pos": [4, 4], "goal": [0, 0]}
]

for step in range(10):
    for drone in drones:
        r, c = drone["pos"]
        gr, gc = drone["goal"]
```

```
        if r < gr:
            r += 1
        elif r > gr:
            r -= 1
        elif c < gc:
            c += 1
        elif c > gc:
            c -= 1

        drone["pos"] = [r, c]

    if drones[0]["pos"] == drones[1]["pos"]:
        print("Conflict detected")
        drones[1]["pos"][0] = max(0, drones[1]["pos"][0] - 1)

    print("Drone positions:", [d["pos"] for d in drones])
```

Application

Each drone attempts to maximize its own navigation reward while dealing with another competing agent.

Analysis

A competitive reward can be defined as:

+10: reaching destination

−10: collision

−1: each movement step

The drones therefore learn to reach their destinations efficiently while avoiding conflicts.

Expected Result

The drones navigate independently and respond to the movements of competing agents.

7. POMDP for Autonomous Vehicle Navigation Under Limited Visibility

Concept

A Partially Observable Markov Decision Process (POMDP) is used when the agent cannot directly observe the complete environment.

For an autonomous vehicle during fog or heavy rain, the vehicle may not know the exact position of another vehicle. It must use observations and maintain a belief about the environment.

Simulation

```
import random

states = ["clear", "vehicle_near", "obstacle"]
observations = ["safe", "danger"]

belief = {
    "clear": 0.6,
    "vehicle_near": 0.3,
    "obstacle": 0.1
}

for step in range(10):
    observation = random.choice(observations)

    if observation == "danger":
```

```
        action = "slow_down"
    else:
        action = "move_forward"

    print("Observation:", observation)
    print("Action:", action)

    if action == "slow_down":
        belief["vehicle_near"] += 0.1
        belief["clear"] -= 0.1

    print("Updated belief:", belief)
```

Application

The vehicle does not have complete information, so it makes decisions based on observations and estimated beliefs.

Analysis

POMDP is more suitable than a normal MDP when sensors provide incomplete or noisy information. A safety-oriented policy should slow down whenever the probability of danger increases.

Expected Result

The vehicle adapts its action according to available observations instead of assuming that the complete environment is known.

8. Ethical Reinforcement Learning with Safety Constraints

Concept

Ethical RL incorporates safety and ethical constraints into the reward function. The agent should maximize performance without violating safety requirements.

Example for an autonomous vehicle:

Reaching destination → positive reward

Collision → large negative reward

Speed violation → negative reward

Safe driving → positive reward

Simulation

```
import random

Q = {
    "safe": {"slow": 0, "fast": 0},
    "danger": {"slow": 0, "fast": 0}
}

for episode in range(100):
    state = random.choice(["safe", "danger"])

    if state == "danger":
        action = "slow"
    else:
        action = random.choice(["slow", "fast"])

    if action == "slow":
        reward = 5
    elif state == "safe" and action == "fast":
        reward = 3
    else:
        reward = -20
```

```
    Q[state][action] += 0.1 * (reward - Q[state][action])

for state in Q:
    print(state, Q[state])
```

Application

The agent is encouraged to choose safe actions when the environment is dangerous.

Analysis

Simply maximizing reward may result in unsafe behavior. Therefore, ethical RL should assign sufficiently high penalties to unsafe actions or use explicit safety constraints. This makes the learned policy safer and more responsible.

Expected Result

The agent learns to select safe actions, particularly when the environment indicates danger.

9. RL-Based Adaptive Traffic Signal Control

Concept

Reinforcement Learning can dynamically control traffic signals according to traffic conditions. The state can represent the number of vehicles waiting at each lane, while the action selects which signal phase should be active.

Simulation

```
import random

Q = {}
```

```
for episode in range(100):
    traffic = (
        random.randint(0, 20),
        random.randint(0, 20)
    )

    action = random.choice(["NS", "EW"])

    if action == "NS":
        reward = traffic[0] - traffic[1]
    else:
        reward = traffic[1] - traffic[0]

    if traffic not in Q:
        Q[traffic] = {"NS": 0, "EW": 0}

    Q[traffic][action] += 0.1 * (
        reward - Q[traffic][action]
    )

for state, values in list(Q.items())[:5]:
    print(state, values)
```

Application

State: traffic waiting in different lanes

Action: select traffic signal phase

Reward: reduction in waiting vehicles

Goal: minimize congestion and waiting time

Analysis

A fixed-time signal treats every traffic condition similarly. RL allows the signal to adapt to changing traffic density. Performance can be evaluated using:

Average waiting time

Queue length

Vehicle throughput

Number of stops

Expected Result

The adaptive controller should learn to give more green time to lanes with higher traffic demand, reducing congestion compared with a fixed strategy.

10. RL-Based Smart Energy Management in a Microgrid

Concept

A smart energy management system uses RL to decide how available energy should be distributed among loads, batteries, renewable sources, and the grid.

Simulation

```
import random

Q = {}

for episode in range(100):
    solar = random.randint(0, 10)
    demand = random.randint(1, 10)

    state = (solar, demand)
    actions = ["battery", "grid", "solar"]
```

```
    action = random.choice(actions)

    if action == "solar":
        reward = min(solar, demand)
    elif action == "battery":
        reward = 0.8 * demand
    else:
        reward = -0.5 * demand

    if state not in Q:
        Q[state] = {a: 0 for a in actions}

    Q[state][action] += 0.1 * (
        reward - Q[state][action]
    )

for state, values in list(Q.items())[:5]:
    print(state, values)
```

Application

State: solar generation and energy demand

Action: use solar, battery, or grid energy

Reward: efficient energy usage and reduced grid dependence

Goal: minimize energy cost and maximize renewable utilization

Analysis

The RL agent learns an adaptive energy policy. When solar energy is sufficient, it should prefer solar power. When demand exceeds generation, battery or grid energy can be selected.

Performance can be evaluated using:

Energy cost

Renewable energy utilization

Grid dependency

Battery usage

Supply-demand balance

Expected Result

The RL system learns energy-management decisions that improve power utilization and reduce unnecessary grid consumption.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively